

5주차_6장(~6.1.4)

6장. 합성곱 신경망 II

6.1 이미지 분류를 위한 신경망

[6.1.1 LeNet-5]

- 합성곱과 다운 샘플링을 반복적으로 거치고, 마지막에 완전연결층에서 분류를 수행

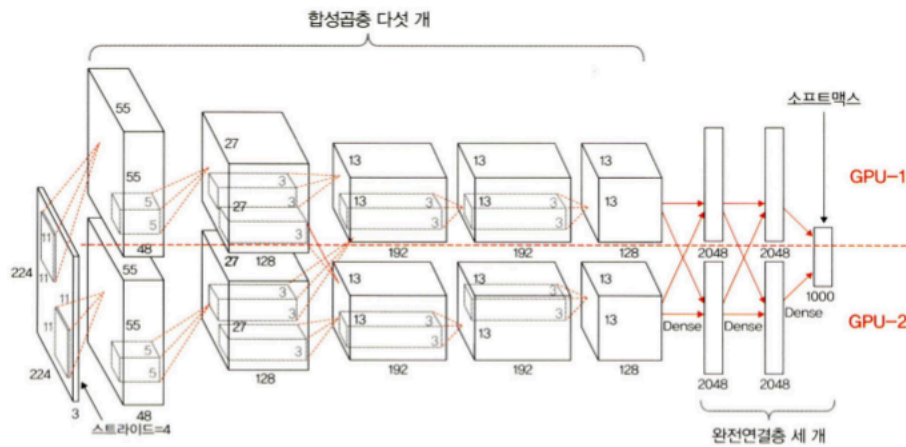
```
# 이미지 전처리
class ImageTransform():
    def __init__():
        self.data_transform={
            'train': transforms.Compose([
                transforms.RandomResizedCrop(resize,scale=(0.5,
1.0)),
                transforms.RandomHorizontalFlip(),
                transforms.ToTensor(),
                transforms.Normalize(mean,std)]),
            'val': transforms.Compose([
                transforms.Resize(256),
                transforms.CenterCrop(resize),
                transforms.ToTensor(),
                transforms.Normalize(mean,std)
            ])
        }

    def __call__(self,img,phase):
        return self.data_transform[phase](img)
```

- `transforms.Compose()` : 이미지 변형 방식들의 묶음

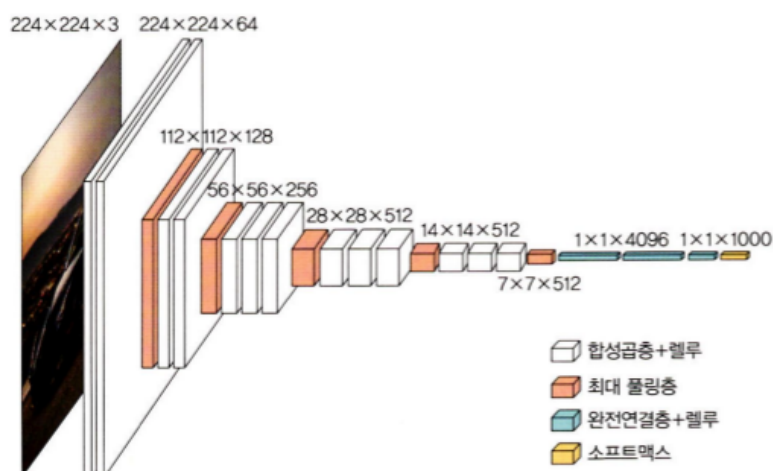
- `__call__` : 인스턴스가 호출되었을 때 실행되는 메서드. 객체 생성 시 return값이 반환 됨.

[6.1.2 AlexNet]



- 합성곱층 5개, 완전연결층 3개로 구성
- 마지막 완전연결층은 카테고리 1000개를 분류하기 위한 소프트맥스 활성화 함수 사용
- GPU 2개를 기반으로 한 병렬 구조를 제외하면 LeNet-5와 차이 없음.

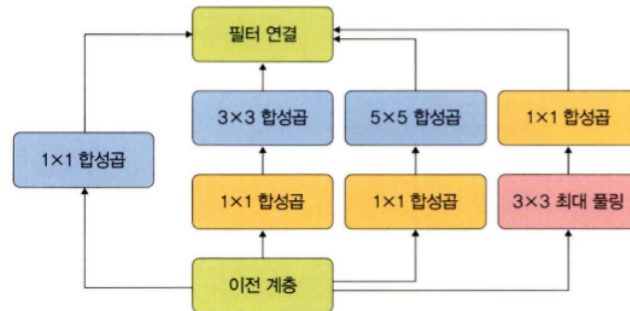
[6.1.3 VGGNet]



- 합성곱층의 파라미터 수를 줄이고 훈련시간을 개선하기 위해 탄생
- 네트워크를 깊게 만드는 것이 성능에 어떤 영향을 미치는지 확인하고자 나온 것
- 총 1억 3300만개 파라미터 존재, 모든 합성곱 커널의 크기는 3×3, 최대 풀링 커널의 크기는 2×2이며, 스트라이드는 2

→ 결과적으로 64개의 224x224 특성 맵 생성 (224x224x64), 마지막 16번째 계층 제외 모두 ReLU 활성화 함수 적용

[6.1.4 GoogLeNet]



- 주어진 하드웨어 자원을 최대한 효율적으로 이용하면서 학습 능력은 극대화할 수 있는 깊고 넓은 신경망
- 깊고 넓은 신경망을 위해 인셉션 모듈 추가
 - 특징을 효율적으로 추출하기 위해 1x1, 3x3, 5x5의 합성곱 연산을 각각 수행
 - 3x3 최대 풀링은 입력과 출력의 높이와 너비가 같아야 하므로 풀링 연산에서는 드물게 패딩을 추가
- 희소 연결 적용
 - 희소 연결 : 뻘뻘하게 연결된 신경망 대신 관련성(correlation)이 높은 노드끼리만 연결 → 연산량 감소 및 과적합 해결
- 4가지 인셉션 모듈
 - 1x1 합성곱
 - 1x1 합성곱 + 3x3 합성곱
 - 1x1 합성곱 + 5x5 합성곱
 - 3x3 최대 풀링(maxpooling) + 1x1 합성곱(convolutional)